

It can also send commands to drivers instructing them to cut their power levels (based on the capabilities that it learned earlier, of course). There is also some traffic the other way. In particular, when a device such as a keyboard or a mouse detects activity after a period of idleness, this is a signal to the system to go back to (near) normal operation.

5.8.3 Application Program Issues

So far we have looked at ways the operating system can reduce energy usage by various kinds of devices. But there is another approach as well: tell the programs to use less energy, even if this means providing a poorer user experience (better a poorer experience than no experience when the battery dies and the lights go out). Typically, this information is passed on when the battery charge is below some threshold. It is then up to the programs to decide between degrading performance to lengthen battery life or to maintain performance and risk running out of energy.

The question is: how can a program degrade its performance to save energy? The answer is application-specific. For instance, a video player that normally plays video in full color at 30 frames/sec could save energy by abandoning the color information and displaying the video in black and white. Another form of degradation is to reduce the frame rate, which leads to flicker and gives the movie a jerky quality. Still another form of degradation is to reduce the number of pixels in both directions, either by lowering the spatial resolution or making the displayed image smaller. Measures of this type saved about 30% of the energy.

Another solution is to alternate between local and remote processing. For instance, we may save power by pushing a computationally expensive operation to the cloud rather than executing it on, say, a smartphone. Whether this is a good idea is a tradeoff between the cost of executing things locally versus the energy cost of operating the radio. Of course, other considerations such as performance (will the delay go up?), security (do we trust the cloud with our computation?), and reliability (what if we do not have connectivity?) also play a role.

There are many other things that applications could do. Of course, they generally mean that the application must be designed with power management in mind. Doing so is especially interesting for battery-powered devices, where accepting some quality degradation means that the user can run longer on a given battery.

5.9 RESEARCH ON INPUT/OUTPUT

Improvement of input/output is an active research domain. Many projects focus solely on efficiency, but there is also much research on other important objectives, such as security or power consumption.

In some cases, the focus is on performance with the aim of improving security. For instance, researchers have tried to accelerate the processing of network processing to build intrusion detection systems that handle the network speeds of modern data center connections (Zhao et al., 2020). Since shuttling data back and forth between the network card and the CPU at 100+ Gbps is hard, they make use of Field Programmable Gate Arrays (FPGAs) to do most of the processing on the network card itself. Other researchers similarly push much of the filtering (selecting specific packets) to the network card's FPGA accelerator, so that only a few relevant packets are processed by the CPU (Brunella et al., 2020).

Pushing processing to processors on the network card is also the goal of the work by Pismenny et al. (2021). However, instead of doing all the processing of the lower layers of the network stack on the card, the authors propose a combined software/NIC architecture where some of the processing takes place on the CPU and the rest on the NIC. All processing that is done on the CPU no longer needs to be done by the card. Others, instead, aggressively optimize the software to ensure as much of the processing as possible can be done from the L1 and L2 caches to allow 100 Gbps processing without coprocessors on the network card (Farshin et al., 2021).

Performance is also important in storage systems. With fast storage devices, the host-level I/O is increasingly a bottleneck for data-intensive computing. The need to improve I/O performance requires optimizations that involve the page cache maintained by the kernel as well as the access to the storage devices (Papa-georgiou et al., 2021). As we have seen, memory mapped I/O is efficient if the file data is in memory, but if not, the data has to be brought in and some existing data evicted. Doing so is expensive, and not flexible (decided once and for all by the kernel's policy).

As in all chapters so far, security is an important concern here also. Unfortunately, researchers have shown that I/O improvements in hardware may offer new opportunities for attackers. A good example is DMA which is good for efficiency, but may allow malicious devices (such as a display cable that has been tampered with) to access memory to which they should not have access (Markettos et al., 2019; Alex et al., 2021). Sometimes a combination of features may create problems. For instance, a CPU feature that allows devices to access caches directly, known as Direct Cache Access, combined with Remote DMA (across the network), allows attackers to launch traditional cache attacks from another machine (Kurth et al., 2020). At the same time, novel CPU features, such as trusted execution environments (TEE), also help to provide stronger security guarantees in I/O by providing I/O libraries inside the TEE (Thalheim et al., 2021).

Finally, power management is a major headache not just for PCs or battery powered devices, but also for large data centers. To help alleviate the pain, data centers use power capping—forcefully limiting the power that a server can use. For instance, if you have 4 MW of power available for your servers and each server can use up to 400 W, you can fit no more than 10,000 servers, even if each server never

really uses more than 300 W. By capping the amount of power each server can draw to 300 W, we can pack an additional 3333 servers in our datacenter. Of course, power capping and increasing workloads make it challenging to ensure low latency for those tasks that need it, while providing sufficient throughput for batch jobs (Li et al., 2020).

5.10 SUMMARY

Input/output is an often neglected, but important, topic. A substantial fraction of any operating system is concerned with I/O. I/O can be accomplished in one of three ways. First, there is programmed I/O, in which the main CPU inputs or outputs each byte or word and sits in a tight loop waiting until it can get or send the next one. Second, there is interrupt-driven I/O, in which the CPU starts an I/O transfer for a character or word and goes off to do something else until an interrupt arrives signaling completion of the I/O. Third, there is DMA, in which a separate chip manages the complete transfer of a block of data, given an interrupt only when the entire block has been transferred.

I/O can be structured in four levels: the interrupt-service procedures, the device drivers, the device-independent I/O software, and the I/O libraries and spoolers that run in user space. The device drivers handle the details of running the devices and providing uniform interfaces to the rest of the operating system. The device-independent I/O software does things like buffering and error reporting.

Secondary storage comes in a variety of types, including magnetic disks, RAIDs, and flash drives. On rotating disks, disk arm scheduling algorithms can often be used to improve performance, but the presence of virtual geometries complicates matters. By pairing two disks or SSDs, a stable storage medium with certain useful properties can be constructed.

Clocks are used for keeping track of the real time, limiting how long processes can run, handling watchdog timers, and doing accounting.

Character-oriented terminals have a variety of issues concerning special characters that can be input and special escape sequences that can be output. Input can be in raw mode or cooked mode, depending on how much control the program wants over the input. Escape sequences on output control cursor movement and allow for inserting and deleting text on the screen.

Most UNIX systems use the X Window System as the basis of the user interface. It consists of programs that are bound to special libraries that issue drawing commands and an X server that writes on the display.

Many personal computers use GUIs for their output. These are based on the WIMP paradigm: windows, icons, menus, and a pointing device. GUI-based programs are generally event driven, with keyboard, mouse, and other events being sent to the program for processing as soon as they happen. In UNIX systems, the GUIs almost always run on top of X.